



Module5—Priority Queue(Heap)



5.0 Priority Queue ADT(คิวแบบมีลำดับความสำคัญ)

- ข้อมูลนามธรรมแบบคิว
- คิวแบบเติมเก็บข้อมูลตามลำดับเวลาที่เข้ามาก่อนหลัง
- เปลี่ยนการจัดเก็บตามลำดับความสำคัญ จากมากไปน้อย
- ข้อมูลที่มีลำดับความสำคัญสูงสุดอยู่ที่หัวคิว
- ข้อมูลที่มีความสำคัญต่ำสุดอยู่ที่ท้ายคิว
- การนำข้อมูลเข้า Priority Queue จะกระทำที่ท้ายคิว เช่นเดียวกับ คิวปกติ

ตัวอย่าง

การให้การรักษาผู้ป่วยที่มีอาการหนักก่อน แม้จะมาทีหลัง



ตัวอย่าง

- การให้การรักษาผู้ป่วยที่มีอาการหนักก่อน แม้จะมาทีหลัง
- การสั่งงานใน printer ส่วนกลาง
- การจัด Process Scheduling ใน Operating system :
Priority queue ใช้ในการเลือก process ถัดไปที่จะประมวลผล
โดยกระบวนการที่มีลำดับความสำคัญสูงกว่าจะถูกเลือกทำงานก่อน

การ Implement(สร้าง) priority queue ในบทนี้จะใช้โครงสร้าง
ข้อมูล แบบ Binary Heap



5.1 Binary Heap : Trees พื้นฐาน

Full Binary Tree

A binary tree in which every node has either 0 or 2 children.

Complete binary tree

A binary tree in which every level, except possibly the deepest, is completely filled, and the nodes at the deepest level are as far left as possible.

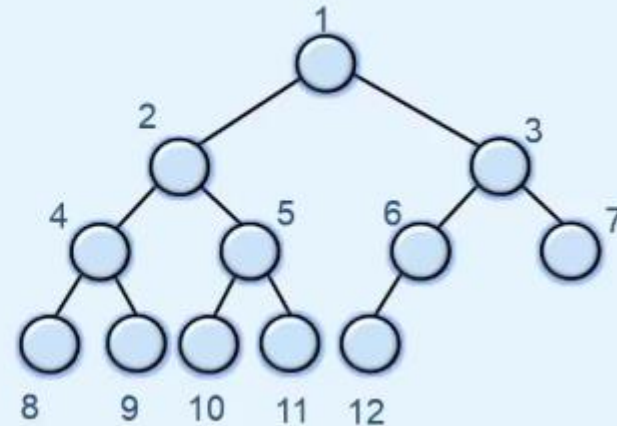
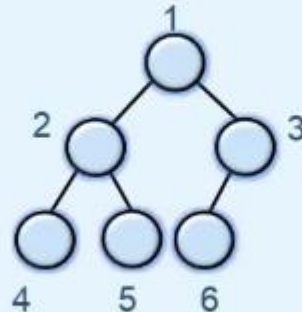
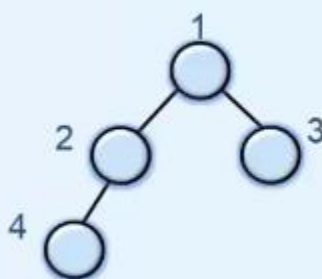


Complete Binary Tree คือ

- ไบนารีทรี
- ที่แต่ละสมาชิกจะต้องมีสมาชิกครบทั้งโนหนดลูกทางซ้าย (Left Child Node) และโนหนดลูกทางขวา (Right Child Node)
- ยกเว้นโนหนดในระดับใบ (Leaves) ที่สามารถมีสมาชิกไม่ครบได้

โนหนดในระดับสุดท้ายจะถูกจัดเรียงจากซ้ายไปขวาจนเต็มเท่าที่จะเป็นไปได้

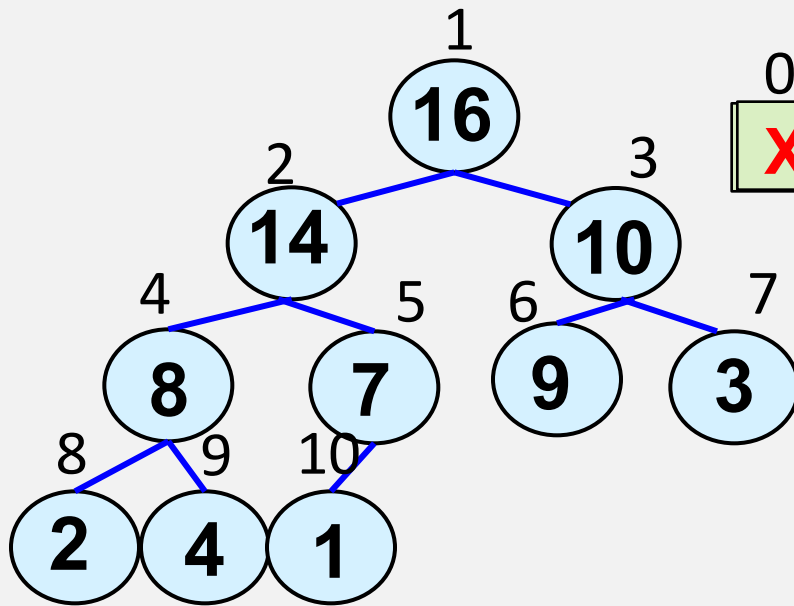
Complete Binary Tree





5.1 Heap

Definition The (binary) heap data structure is an array object that can be viewed as a complete binary tree. Each node of the tree corresponds to an element of the array that stores the value in the node.



0	1	2	3	4	5	6	7	8	9	10
X	16	14	10	8	7	9	3	2	4	1

ช่องที่ 0 ไม่ใช่เก็บข้อมูล
จะทำให้เขียนโปรแกรมง่าย



A is an array represent heap attribute

- int A[30];

30

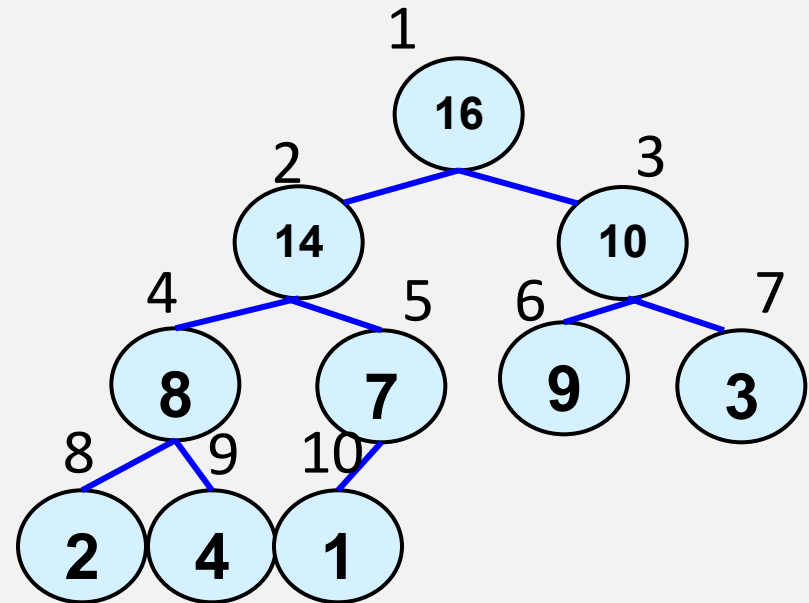
- length ขนาดของ A

10

- heapsize จำนวนสมาชิกใน heap

- heapsize $\leq$ length

0	1	2	3	4	5	6	7	8	9	10	..
X	16	14	10	8	7	9	3	2	4	1	





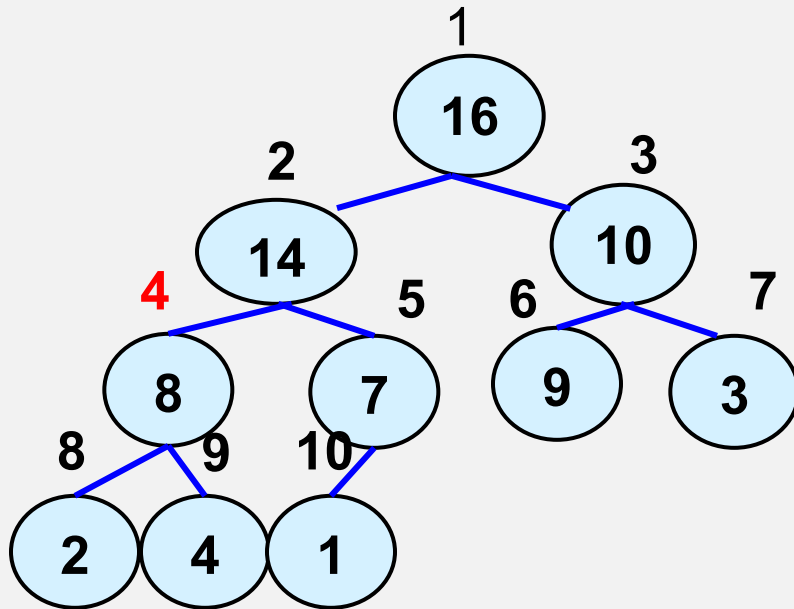
ตอบคำถาม

1. Priority Queue คืออะไร
2. Heap คืออะไร
3. ถ้ากำหนด `int A[30];` เก็บ heap
ค่า 30 คือ length หรือ heapsize??



5.1.1 การไปยัง parent และ child แบบทรี ขณะเก็บข้อมูลด้วย Array

	1	2	3	4	5	6	7	8	9	10
X	16	14	10	8	7	9	3	2	4	1

 $i=4$ 

parents(i)

Parents = 2

return $i/2$

left(i)

Left = 8

return $2i$

right(i)

right = 9

return $2i+1$



	1	2	3	4	5	6	7	8	9	10
X	16	14	10	8	7	9	3	2	4	1

$i=2$

parent คือ?

left คือ ?

right คือ ?

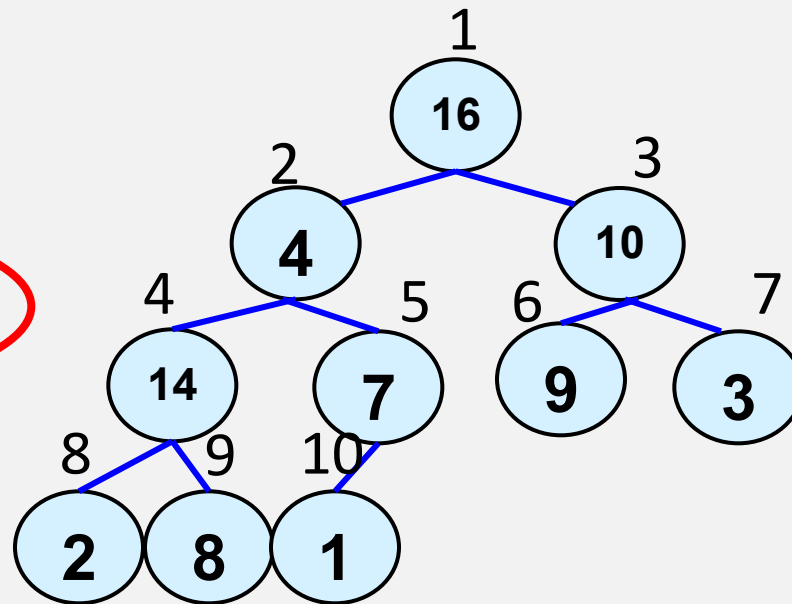


5.1.2 Heap property :

Heaps also satisfy the heap property: for every node i other than the root,

$$A[\text{parent}(i)] \geq A[i]$$

ตัวใดที่ขาด
คุณสมบัติ?





**การเปลี่ยน Array ธรรมดาให้เป็น heap ด้วย
algorithm heapify**



5.1.2 Maintaining the heap property

Heapify(A,i)

$l = \text{left}(i)$

$r = \text{right}(i)$

if $l \leq \text{heapsize}$ and $A[l] > A[i]$

then $\text{largest} = l$

else $\text{largest} = i$

if $r \leq \text{heapsize}$ and $A[r] > A[\text{largest}]$

then $\text{largest} = r$

if $\text{largest} \neq i$

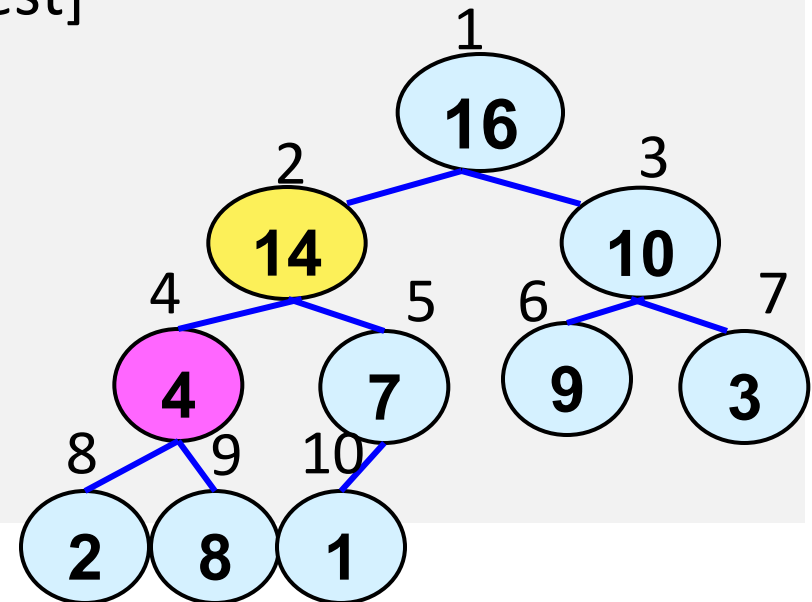
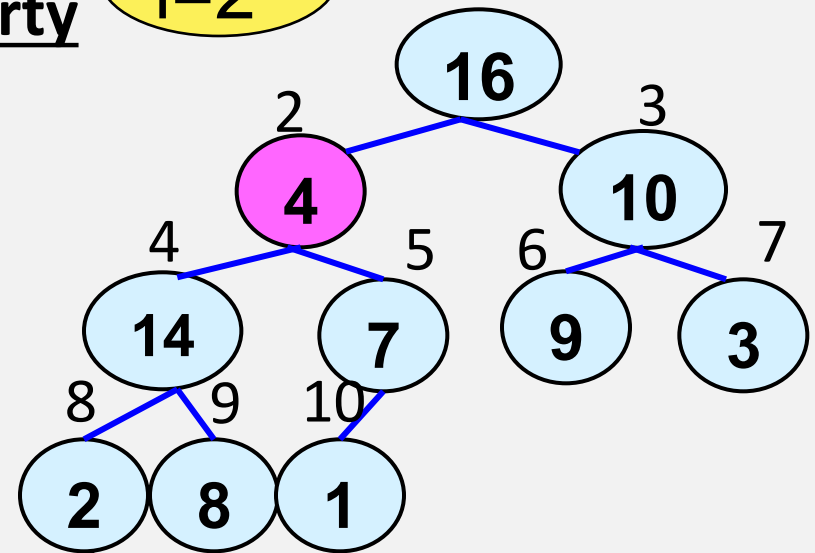
then

4, 14

exchange(A[i], A[largest])

Heapify(A, largest)

i=2





Heapify(A,i)

$l = \text{left}(i)$

$r = \text{right}(i)$

if $l \leq \text{heapsize}$ and $A[l] > A[i]$

then $\text{largest} = l$

else $\text{largest} = i$

if $r \leq \text{heapsize}$ and $A[r] > A[\text{largest}]$

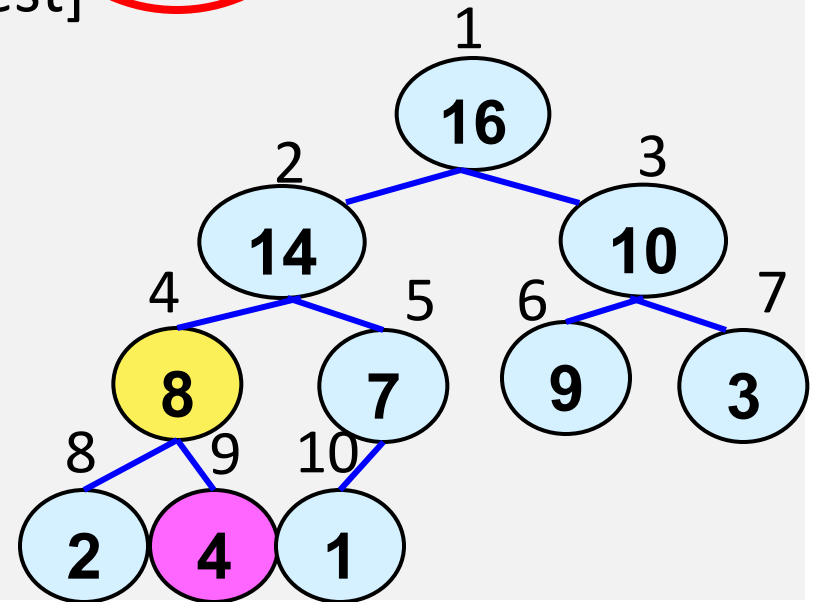
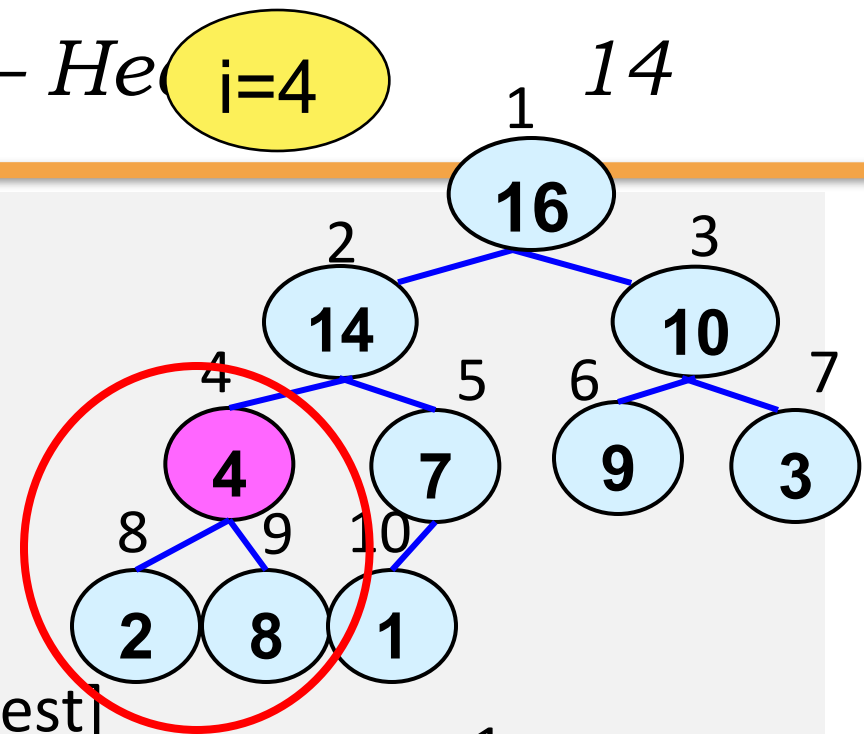
then $\text{largest} = r$

if $\text{largest} \neq i$

then

exchange($A[i], A[\text{largest}]$)

Heapify(A, largest)





Heapify(A,i)

l=left(i)

r=right(i)

if $l \leq \text{heapsize}$ and $A[l] > A[i]$

then largest = l

else largest = i

if $r \leq \text{heapsize}$ and $A[r] > A[\text{largest}]$

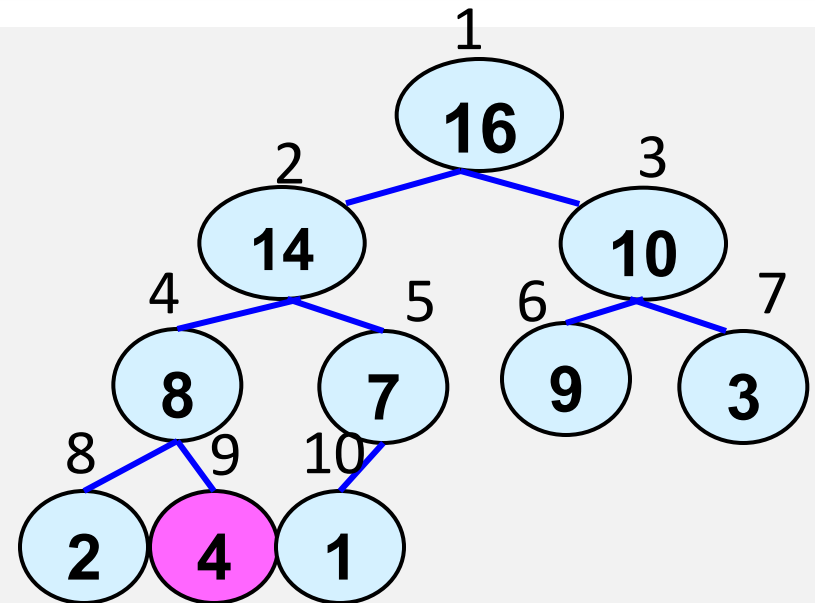
then largest = r

if largest $\neq i$

then

exchange($A[i], A[\text{largest}]$)

Heapify(A, largest)

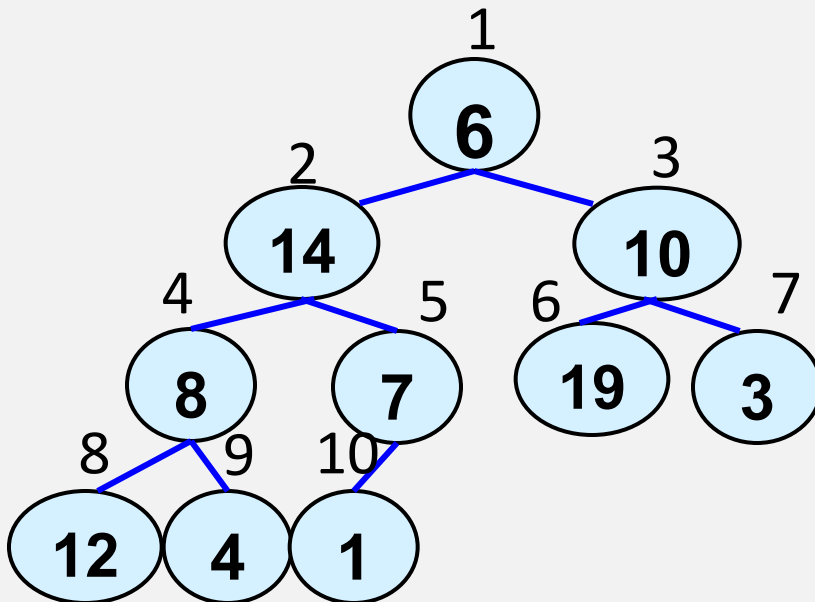


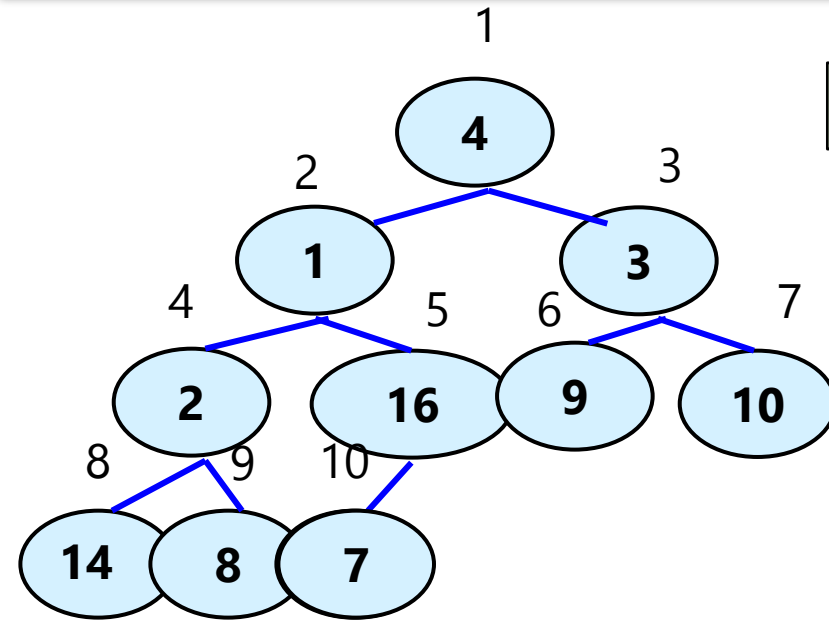


algorithm heapify เป็นการสร้าง array ให้เป็น heap ที่ละโหนด

คำถาม

ถ้า heapify $i=1$ แล้ว array นี้จะมีคุณสมบัติเป็น heap ทั้ง array หรือไม่





0	1	2	3	4	5	6	7	8	9	10
X	4	1	3	2	16	9	10	14	8	7
X	4	1	3	14	16	9	10	2	8	7
X										
X										
X										

Build_heap(A)

```
{ length=heapsize
  for(i= length/2 downto 1)
    Heapify(A,i)
}
```

Build heap มี bigO เท่าใด??



โครงสร้าง	Insert	Min/Max	Delete
Priority Q (Binary Heap)	$O(\log n)$	$O(1)$	$O(\log n)$

คำถาม

- 1.Heapify 1 ค่า มี bigO เท่าใด
- 2.Build head มี bigO เท่าใด



ความสูง h	จำนวน node	Heapify สูงสุด
0	$n/2$	0
1	$n/4$	1
2	$n/8$	2
3	$n/16$	3
...	...	...

$$T(n) = \frac{n}{4}(1) + \frac{n}{8}(2) + \frac{n}{16}(3) + \dots$$



$$T(n) = n \left(\frac{1}{4} + \frac{2}{8} + \frac{3}{16} + \dots \right)$$



$$\sum_{h=1}^{\infty} \frac{h}{2^{h+1}} = 1$$



$$T(n) \leq n$$



$$\text{BUILD-HEAP} = O(n)$$



- เมื่อ Array มีคุณสมบัติเป็น heap ข้อมูลตัวที่มากที่สุดจะอยู่ด้านหน้าสุด
- สามารถนำคุณสมบัตินี้มาใช้ในการเรียงข้อมูลในอะเรย์ เช่นการเรียงข้อมูลจากน้อยไปมาก



ต้องการเรียงข้อมูลจากน้อยไปมาก

X	7	5	4	3
---	---	---	---	---

Heap

X	3	5	4	7
---	---	---	---	---

X	5	3	4	7
---	---	---	---	---

Heap

X	4	3	5	7
---	---	---	---	---

X	4	3	5	7
---	---	---	---	---

Heap

X	3	4	5	7
---	---	---	---	---



5.1.4 Heap sort :

Heapsort(A)

Build_heap(A)

for i = length down to 2
do

exchange (A[1], A[i])

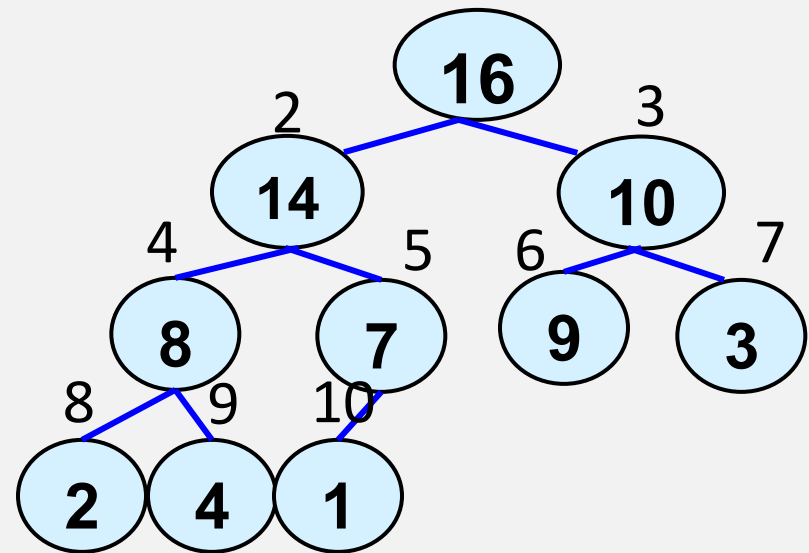
heapsize = heapsize - 1;

Heapify(A,1)

	1	2	3	4	5	6	7	8	9	10
X	16	14	10	8	7	9	3	2	4	1

	1	2	3	4	5	6	7	8	9	10
X	1	14	10	8	7	9	3	2	4	16

i=10





Heapsort(A)

Build_heap(A)

for i = length down to 2
do

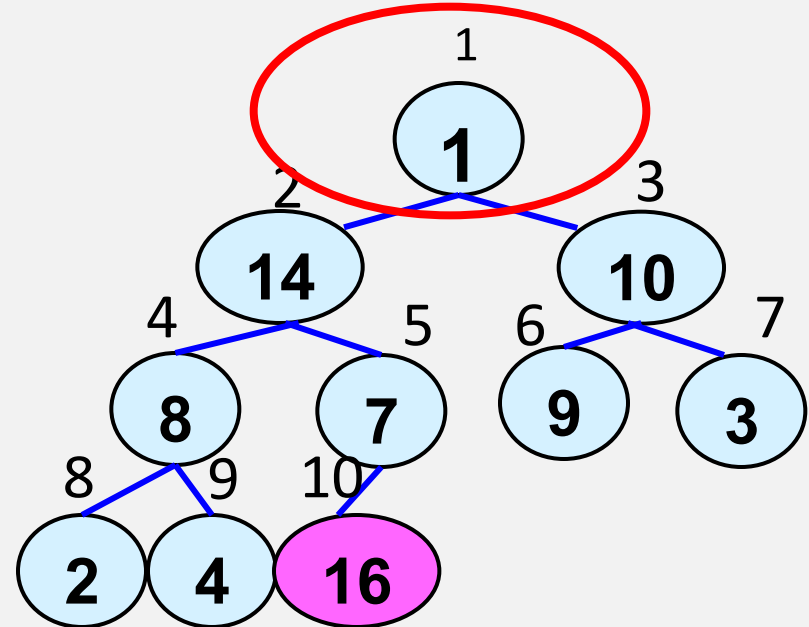
exchange (A[1], A[i]) **ต่อ**

heapsize = heapsize - 1;

Heapify(A,1)

A[1-9]

	1	2	3	4	5	6	7	8	9	10
	X	1	14	10	8	7	9	3	2	4
										16





Heapsort(A)

Build_heap(A) $i=9$

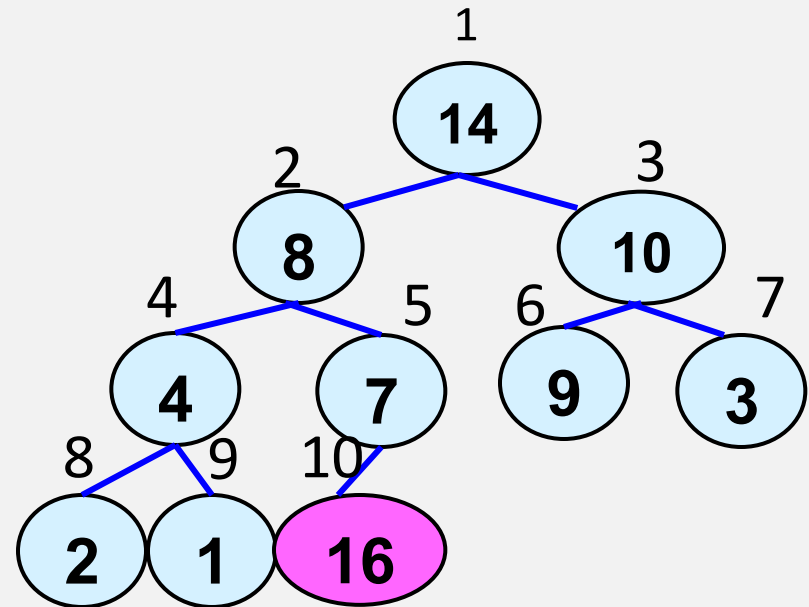
for $i = \text{length}$ down to 2
do

exchange (A[1], A[i])

heapsize = heapsize - 1;

Heapify(A,1)

1	2	3	4	5	6	7	8	9	10
X	14	8	10	4	7	9	3	2	16





Heapsort(A)

Build_heap(A)

for i = length down to 2

do

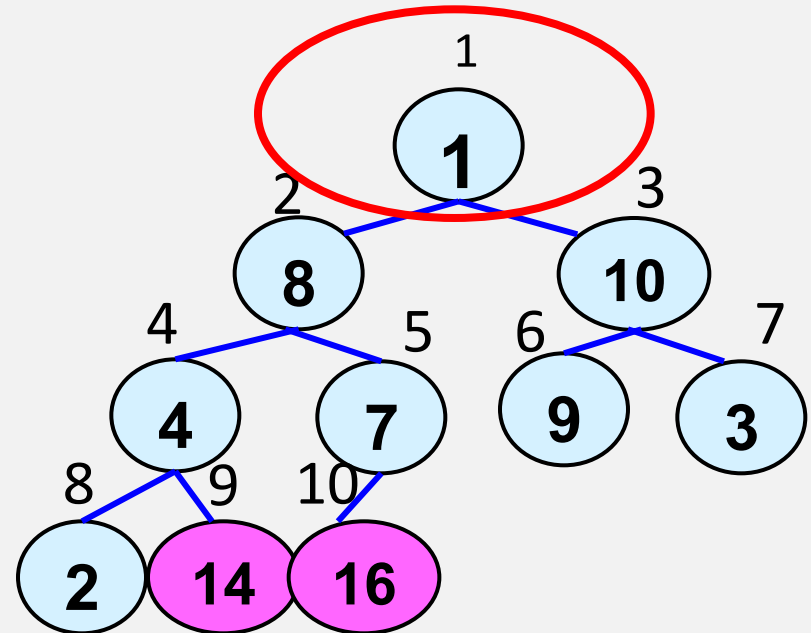
exchange (A[1], A[i])

heapsize = heapsize -1;

Heapify(A,1)

A[1-8]

	1	2	3	4	5	6	7	8	9	10
	X	1	8	10	4	7	9	3	14	16





5.1.5 Priority queues

Priority queues : is a data structure for maintaining a set S of elements, each with a associated value called a key. A priority supports the following operations.

- 1) Insert(S, x) : insert the element x into the set S . This operation could be written as $S \leftarrow S \cup \{ x \}$
- 2) Maximum(S) : returns the elements of S with the largest key;
- 3) Extract_Max(S) : return the elements of S with the largest key.



Priority queues operators :

1. Insert $O(\log n)$
2. Maximum $O(1)$
3. Extract_Max ??



45

	1	2	3	4	5	6	7	8	9	10	11
X	16	14	10	8	7	9	3	2	4	1	7
X	16	14	10	8	45	9	3	2	4	1	7

Heap_Insert(A, key)

heapsize = heapsize+1

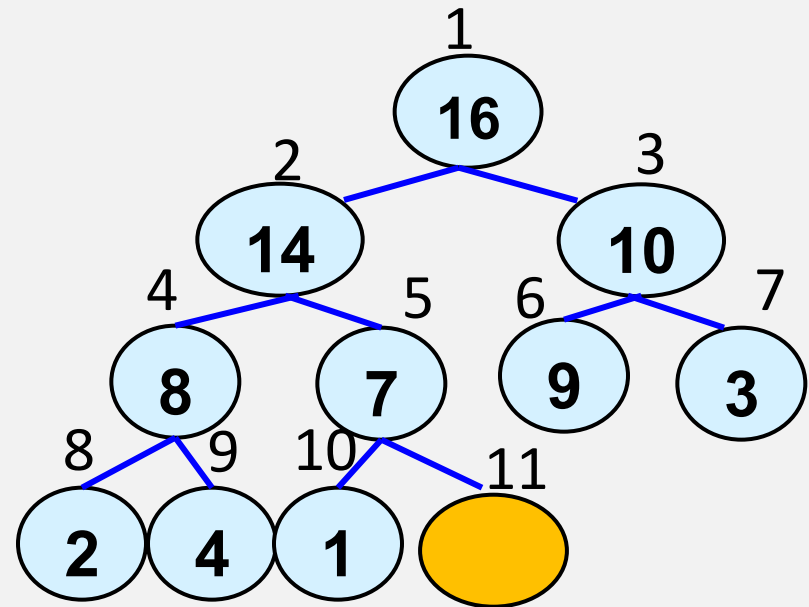
i = heapsize

while i > 1 && A[parent(i)] < key

A[i] = A[parent(i)]

i = parent(i)

A[i] = key





45

	1	2	3	4	5	6	7	8	9	10	11
X	16	14	10	8	7	9	3	2	4	1	
X	16	14	10	8	45	9	3	2	4	1	7
X	16	45	10	8	14	9	3	2	4	1	7
X	45	16	10	8	14	9	3	2	4	1	7



Heap_Exact_Max(A)

if heap_size < 1

then error “Heap underflow”

max = A[1]

max=45

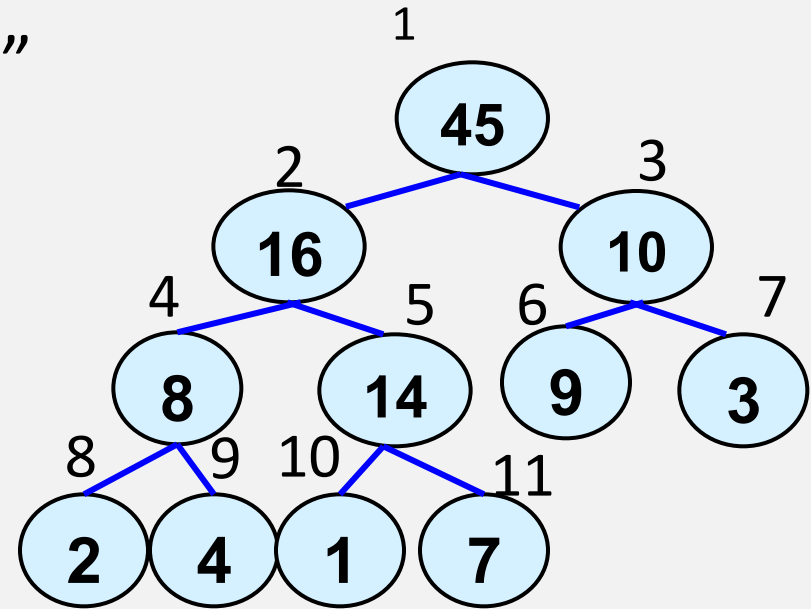
A[1] = A[heapsize]

heapsize = heapsize-1

Heapify(A,1)

return max

	1	2	3	4	5	6	7	8	9	10	11
X	45	16	10	8	14	9	3	2	4	1	7





Heap_Exact_Max(A)

if heap_size < 1

then error “Heap underflow”

max = A[1]

max=45

A[1] = A[heapsize]

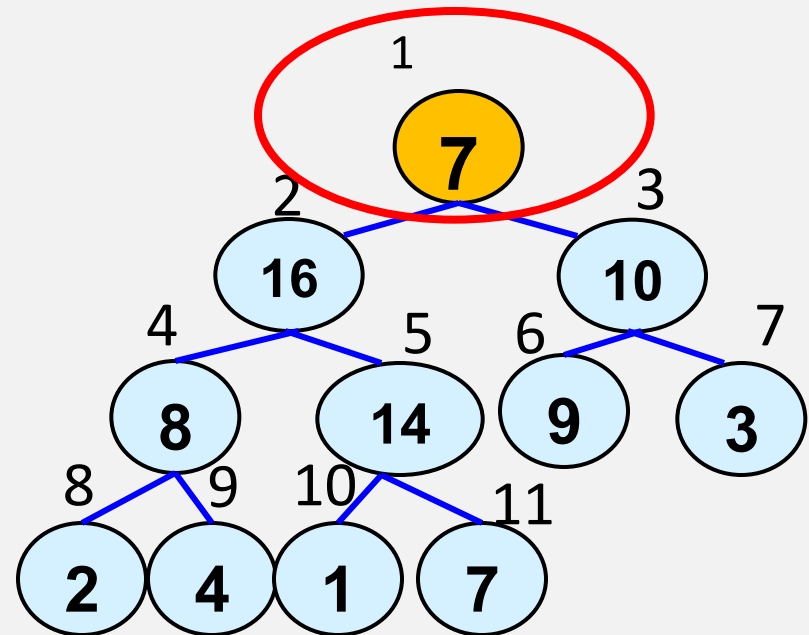
heapsize = heapsize-1

Heapify(A,1)

return max

A[1-10]

	1	2	3	4	5	6	7	8	9	10	11	
Initial Array	X	45	16	10	8	14	9	3	2	4	1	7
After Swap	X	7	16	10	8	14	9	3	2	4	1	7





Heap_Exact_Max(A)

if heap_size < 1

then error “Heap underflow”

max = A[1]

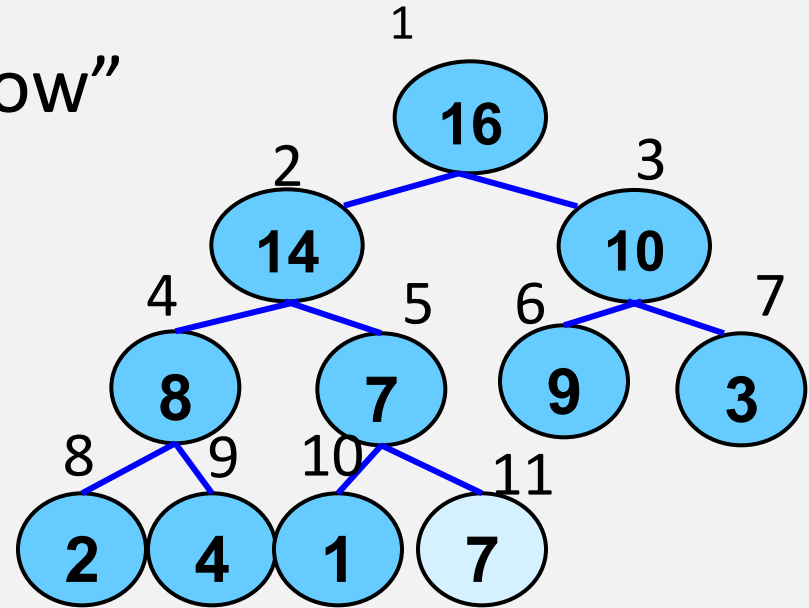
A[1] = A[heapsize]

heapsize = heapsize-1

Heapify(A,1)

return max

	1	2	3	4	5	6	7	8	9	10	11
X	16	14	10	8	7	9	3	2	4	1	7



max=45